

Documento de Diseño de Software (SDD)

Estándar IEEE 1016 — Software Design Description

Sistema de Procesamiento de Pagos con Módulo de Anonimización

Campo	Detalle
Estudiante	[Nombre y apellido]
Curso / Materia	[Nombre de la materia]
Docente	[Nombre del docente]
Módulo	1 — Arquitectura Estructural UML del Payment Switch
Fecha de entrega	[DD/MM/AAAA]
Versión del documento	1.0

Plantilla de referencia — así se presenta profesionalmente un SDD generado a partir de un prompt de IA: con portada, índice, y las secciones exigidas por el ejercicio, revisadas y editadas por el propio estudiante antes de la entrega.

Índice

Índice	2
1. Resumen Ejecutivo	3
1.1 Propósito	3
1.2 Alcance	3
2. Diccionario de Clases	3
2.1 Card	3
2.2 Transaction	4
2.3 PaymentSwitch	4
2.4 ITokenizationEngine («interface»)	4
2.5 TokenizationService	5
2.6 CryptoHSMSERVICE	5
2.7 Token	5
3. Análisis de Relaciones	6
4. Sección de Seguridad y Anonimización	6
4.1 Cómo protege este diseño al PAN	6
4.2 Cómo protege este diseño al CVV	7
4.3 Cumplimiento con PCI-DSS v4.0	7
5. Guía de Explicación para Evaluación	7
Anexo A — Código Fuente del Diagrama (Mermaid.js)	9

1. Resumen Ejecutivo

1.1 Propósito

Este documento describe el diseño estructural de un sistema de procesamiento de pagos, con énfasis particular en el subsistema SecurityZone, responsable de anonimizar los datos sensibles de la tarjeta antes de que cualquier otro componente del sistema pueda acceder a ellos. El objetivo del diseño es garantizar que el número de tarjeta (PAN) y el código de seguridad (CVV) nunca circulen en texto plano fuera del límite estrictamente necesario para su captura inicial.

1.2 Alcance

El diseño cubre siete elementos, organizados en dos capas:

- **Núcleo transaccional** (fuera del namespace SecurityZone): Card, Transaction y PaymentSwitch, que modelan el flujo de negocio de una transacción de pago.
- **Zona de seguridad** (namespace SecurityZone): ITokenizationEngine, TokenizationService, CryptoHSMSERVICE y Token, que modelan el subsistema de anonimización propiamente dicho.

Quedan fuera del alcance de este diagrama el enrutamiento hacia redes externas (VISA/ISO 8583) y la persistencia en base de datos, que corresponden a vistas complementarias del sistema (comportamental y de despliegue, respectivamente).

2. Diccionario de Clases

2.1 Card

Atributo / Método	Tipo	Justificación
panMasked	String	Representación enmascarada del PAN (p. ej. "411111*****1111"), pensada para mostrarse en interfaces y logs. Es texto por definición: nunca se opera aritméticamente sobre ella.
encryptedPAN	String	El PAN real, cifrado mediante FPE (Format-Preserving Encryption). Se tipa como String porque el cifrado FPE preserva el formato numérico original codificándolo en una cadena.
cvvHash	String	Salida de una función hash (HMAC-SHA256). Un hash es, por naturaleza, una cadena hexadecimal — no existe un tipo numérico apropiado ni tiene sentido operarlo como número.
isValid(): boolean	—	Valida la integridad estructural de la tarjeta (formato, checksum de Luhn) sin necesidad de acceder al PAN en claro.

2.2 Transaction

Atributo / Método	Tipo	Justificación
transactionId	String	Identificador único (típicamente un UUID). Se tipa como String para admitir formatos alfanuméricos estándar sin riesgo de desbordamiento.
amount	BigDecimal	Decisión crítica: nunca debe usarse float o double para representar dinero, ya que ambos introducen errores de redondeo en aritmética de punto flotante que son inaceptables en un sistema financiero. BigDecimal garantiza precisión decimal exacta.
timestamp	DateTime	Marca temporal de la transacción, necesaria tanto para trazabilidad como para el cálculo de cumplimiento de SLA.
status	String	Estado del ciclo de vida de la transacción (p. ej. PENDING, APPROVED, DECLINED). Se modela como String (o enumeración simple) por legibilidad directa en auditorías.
validate(): boolean	—	Verifica reglas de negocio (monto positivo, moneda soportada) antes de que la transacción avance en el flujo.

2.3 PaymentSwitch

Atributo / Método	Tipo	Justificación
switchId	String	Identificador de instancia, relevante en topologías con múltiples switches activos en paralelo.
processTransaction(tx): boolean	—	Único punto de entrada del flujo transaccional. Orquesta el proceso, pero —como se justifica en la Sección 3— no implementa la lógica de tokenización: la delega.

2.4 ITokenizationEngine («interface»)

Contrato abstracto que define qué debe poder hacer cualquier motor de tokenización, independientemente del proveedor de cifrado subyacente.

Método	Justificación
tokenize(rawPan): Token	Recibe el PAN en claro —la única superficie de exposición controlada del sistema— y retorna un objeto Token seguro.
detokenize(tokenValue): String	Operación inversa, reservada a contextos estrictamente autorizados (p. ej. procesos de reembolso).
maskPan(rawPan): String	Genera la representación enmascarada para uso en interfaces y logs, sin necesidad de pasar por el HSM.

2.5 TokenizationService

Atributo / Método	Tipo	Justificación
saltKey	String	Clave secundaria utilizada en el HMAC del CVV. Se declara private porque ninguna otra clase del sistema debe tener acceso directo a ella.
tokenize(), maskPan()	—	Implementación concreta del contrato ITokenizationEngine.
hashCVV(rawCvv): String	—	Método adicional, fuera del contrato de la interfaz, porque hashear el CVV es conceptualmente distinto de tokenizar el PAN: es una operación de un solo sentido (no reversible), mientras que la tokenización sí admite una operación inversa (detokenize).

2.6 CryptoHSMService

Atributo / Método	Tipo	Justificación
hsmKeySlot	int	Referencia al slot de clave dentro del hardware físico del HSM. Un entero simple es suficiente, ya que la clave criptográfica real nunca sale del dispositivo.
encryptFPE(), decryptFPE()	—	Únicos métodos del sistema que ejecutan criptografía fuerte. Aislar esta responsabilidad en una sola clase permite auditar el cifrado sin revisar el resto del código base.

2.7 Token

Atributo / Método	Tipo	Justificación
tokenValue	String	Sustituto no reversible del PAN por sí solo: revertirlo requiere, además, el servicio de detokenización y el HSM.
tokenExpiry	DateTime	Los tokens de este diseño tienen vida corta por decisión de diseño, lo que reduce la ventana de exposición en caso de filtración.
isValid(): boolean	—	Verifica la expiración del token antes de permitir cualquier uso.

3. Análisis de Relaciones

Relación del diagrama	Tipo UML	Justificación técnica
Transaction "1" o-- "1" Card : requiere	Agregación	Una Card existe con independencia de cualquier transacción concreta: fue emitida por un banco mucho antes y seguirá existiendo después. Transaction la usa, no la posee ni controla su ciclo de vida — por eso es agregación y no composición. Si fuera composición, destruir la transacción debería destruir la tarjeta, lo cual no tiene sentido de negocio.
Card "1" o-- "1" Token : vinculada a	Agregación	El Token es administrado por TokenizationService y tiene su propio ciclo de vida (expira y se renueva de forma independiente). Card lo referencia, pero no lo controla de forma exclusiva.
ITokenizationEngine < .. TokenizationService : implementa	Realización	TokenizationService cumple el contrato de la interfaz. Se modela como realización (línea discontinua) y no como herencia porque ITokenizationEngine no aporta estado ni comportamiento propio — es un contrato puro.
TokenizationService ..> CryptoHSMSERVICE : usa HSM	Dependencia	TokenizationService invoca al HSM dentro del cuerpo de sus métodos, pero no lo mantiene como atributo persistente ni es responsable de su ciclo de vida. Es la relación más débil, adecuada para una llamada de servicio puntual.
TokenizationService ..> Token : crea	Dependencia	La instanciación de Token ocurre dentro de un método de TokenizationService, pero el objeto creado sale de ese método y pasa a ser responsabilidad de quien lo recibe (Card).
PaymentSwitch ..> ITokenizationEngine : solicita token	Dependencia	Decisión de diseño central: PaymentSwitch depende de la interfaz, nunca de TokenizationService directamente. Aplicación directa del Principio de Inversión de Dependencias (la “D” de SOLID): permite sustituir el proveedor de tokenización sin modificar PaymentSwitch.

4. Sección de Seguridad y Anonimización

4.1 Cómo protege este diseño al PAN

El PAN en claro solo existe en memoria durante la ejecución de `TokenizationService.tokenize()`, que inmediatamente delega el cifrado fuerte a `CryptoHSMSERVICE.encryptFPE()`. A partir de ese punto, el resto del sistema opera exclusivamente sobre:

- **Token.tokenValue** — el sustituto seguro y no reversible por sí mismo.
- **Card.panMasked** — la representación enmascarada, exclusiva para visualización.
- **Card.encryptedPAN** — el PAN cifrado en reposo mediante FPE.

PaymentSwitch, que orquesta toda la transacción, nunca tiene una referencia directa a CryptoHSMService ni a un PAN en claro: su única vía hacia datos sensibles pasa por la interfaz ITokenizationEngine. Esto significa que un error de programación en PaymentSwitch no puede, estructuralmente, provocar una fuga de PAN en claro.

4.2 Cómo protege este diseño al CVV

El CVV se procesa exclusivamente mediante TokenizationService.hashCVV(), que aplica HMAC-SHA256 con la clave saltKey. Es importante notar que ninguna clase del diagrama posee un atributo para almacenar el CVV original — ni siquiera cifrado. Esa ausencia deliberada es, en sí misma, la evidencia de cumplimiento: el diseño no ofrece ningún lugar donde ese dato podría persistirse por error.

4.3 Cumplimiento con PCI-DSS v4.0

Dato sensible	Técnica aplicada	Estado de persistencia	Requisito PCI-DSS
PAN	Tokenización + cifrado FPE vía HSM	Solo Token.tokenValue persistido; encryptedPAN cifrado en reposo	Req. 3.5
CVV	Hash HMAC-SHA256 con saltKey	Prohibido persistir bajo cualquier forma, incluso cifrada	Req. 3.3.1
Datos en logs/UI	Enmascaramiento (maskPan)	Solo panMasked visible fuera de la zona de seguridad	Req. 3.4
Claves criptográficas	Gestionadas dentro del HSM (hsmKeySlot)	Nunca expuestas a la capa de aplicación	Req. 3.6 / 3.7

La fila del CVV está marcada como prohibición dura y no como mitigación: a diferencia del PAN (que sí puede tokenizarse y usarse de forma controlada), el estándar PCI-DSS v4.0 prohíbe conservar el CVV bajo cualquier forma una vez completada la autorización, sin excepción.

5. Guía de Explicación para Evaluación

Puntos clave para la defensa oral del trabajo:

1. ¿Por qué Transaction–Card es agregación y no composición?

Porque el ciclo de vida de la tarjeta es independiente de cualquier transacción individual. La composición implicaría que la tarjeta se crea y se destruye junto con la transacción, lo cual contradice la realidad de negocio: una tarjeta participa en miles de transacciones a lo largo de su vida útil.

2. ¿Por qué PaymentSwitch depende de ITokenizationEngine y no de TokenizationService?

Es una aplicación del principio de inversión de dependencias: al depender de una abstracción y no de una implementación concreta, el sistema puede cambiar de proveedor de tokenización sin tocar el componente orquestador ni sus pruebas asociadas.

3. ¿Dónde exactamente se garantiza que el CVV no se almacena?

En la ausencia misma de un atributo para el CVV en Card (solo existe cvvHash, que es un hash de un solo sentido) y en que hashCVV() retorna un String sin ninguna ruta de persistencia del valor original.

4. ¿Qué pasa si el HSM falla?

Al estar aislado en una única clase (CryptoHSMService), una falla del HSM se propaga como una excepción controlada desde un único punto del sistema, en lugar de fallar de forma impredecible en múltiples lugares.

5. ¿Por qué Token tiene tokenExpiry?

Es una decisión de defensa en profundidad: aunque el token no es reversible sin el HSM y el servicio de tokenización, limitar su vida útil reduce la ventana de tiempo en la que un token filtrado podría ser mal utilizado.

Anexo A — Código Fuente del Diagrama (Mermaid.js)

Se incluye el código fuente exacto sobre el que se generó este documento, como buena práctica de trazabilidad entre el diagrama y su documentación.

```
classDiagram
    namespace SecurityZone {
        class ITokenizationEngine {
            <<interface>>
            +tokenize(rawPan) Token
            +detokenize(tokenValue) String
            +maskPan(rawPan) String
        }
        class TokenizationService {
            -String saltKey
            +tokenize(rawPan) Token
            +maskPan(rawPan) String
            +hashCVV(rawCvv) String
        }
        class CryptoHSMService {
            -int hsmKeySlot
            +encryptFPE(data) String
            +decryptFPE(encryptedData) String
        }
        class Token {
            -String tokenValue
            -DateTime tokenExpiry
            +isValid() boolean
        }
    }

    class Card {
        -String panMasked
        -String encryptedPAN
        -String cvvHash
        +isValid() boolean
    }

    class Transaction {
        -String transactionId
        -BigDecimal amount
        -DateTime timestamp
        -String status
        +validate() boolean
    }

    class PaymentSwitch {
        -String switchId
        +processTransaction(tx) boolean
    }

    ITokenizationEngine <|.. TokenizationService : implementa
    TokenizationService ..> CryptoHSMService : usa HSM
    TokenizationService ..> Token : crea
    Card "1" o-- "1" Token : vinculada a
    PaymentSwitch ..> ITokenizationEngine : solicita token
    Transaction "1" o-- "1" Card : requiere
```